

Homework 05

AUTHOR
cgao19

PUBLISHED
September 27, 2026

All work is in Python. Seeds are set per simulation question and reported there: **432** for Q1, **433** for Q2, **434** for Q3 and **435** for its diagnostic. Q4 uses no random numbers. Neighbor weights are equal throughout, no intercept column is added to any matrix used to measure distance, and plot formatting is left at library defaults until Q5.

► Show the workflow

Question 1: Estimate the bias and variance of KNN

Part (a)

$f(x_0) = 0.5(0.5) + \sin(0.7) - 0.3(1)^2 = \sin(0.7) - 0.05$. For a fixed target point the whole grid of k comes from one sorted distance vector: sort the training rows by distance to x_0 , take the cumulative sum of the sorted responses, and divide entry k by k . The sort is stable, so distance ties fall to the smaller training row number.

► Show the workflow

```
seed = 432; f(x0) = 0.5942177; k = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29]
one dataset, KNN prediction at x0 by k: 0.666 0.621 0.517 0.475 0.414 0.402 0.413 0.355 0.373
0.378 0.416 0.439 0.459 0.427 0.442
```

A single realization is far too noisy to read a trade-off from, which is what part (b) fixes.

Part (b)

► Show the workflow

k	1	3	5	7	9	11	13	15	17	19	21	23	25	27	29
bias	-0.0446	-0.0368	-0.0442	-0.0537	-0.0621	-0.0645	-0.0672	-0.0717	-0.0774	-0.0859	-0.0901	-0.0974	-0.1041	-0.1085	-0.1100
squared bias	0.0020	0.0014	0.0020	0.0029	0.0039	0.0042	0.0045	0.0051	0.0060	0.0074	0.0081	0.0095	0.0108	0.0117	0.0121
variance	0.0351	0.0194	0.0150	0.0120	0.0109	0.0098	0.0098	0.0088	0.0081	0.0077	0.0078	0.0071	0.0072	0.0072	0.0072
MSE	0.0371	0.0208	0.0170	0.0149	0.0147	0.0140	0.0143	0.0139	0.0141	0.0151	0.0159	0.0166	0.0180	0.0187	0.0191

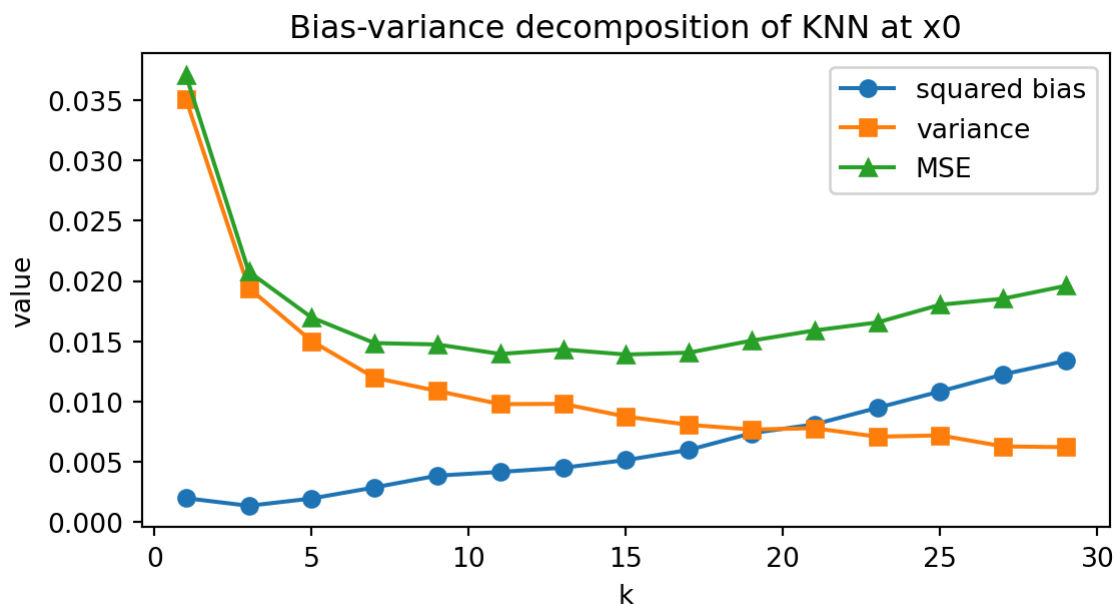
◀  ▶

largest $|MSE - (squared\ bias + variance)|$: $5.20e-17$

MSE smallest at $k = 15$ (MSE = 0.0139); variance at $k = 1$ is 0.0351 against a noise part σ^2 of 0.01

The identity holds to machine precision, as it must: with divisor 200 both terms are computed against the same empirical mean, so it is an algebraic rearrangement rather than an approximation. Divisor 199 would break it visibly.

► Show the workflow



Question 1 figure, library defaults (revised in Question 5).

Variance falls steeply to about $k = 7$ and then declines slowly; squared bias is flat for the first few k and then grows steadily. The sum is U-shaped with a minimum at $k = 15$, though the MSE values between $k = 11$ and $k = 17$ differ by less than 0.0005, so 200 repetitions locate only the region of the minimizer. Two details are worth naming. First, the variance is not the textbook σ^2/k : at $k = 1$ the noise contributes only 0.01 but the measured variance is about 0.035. The rest is design variability, since different training points land near x_0 across repetitions, so the neighborhood average of f moves before any ϵ_i is added. Second, the bias is negative everywhere and grows in magnitude: x_0 sits away from the mode of the design, so a widening neighborhood fills in preferentially toward the origin, where $f(\mathbf{0}) = 0$ is well below $f(x_0) \approx 0.594$, and the concavity of $-0.3x_3^2$ pushes the same way.

Comparison with ridge and lasso. In all three, one knob buys variance reduction with bias and the optimum is interior, with large k and large λ both meaning more smoothing; the correspondence runs through degrees of freedom, about n/k against the trace of the ridge hat matrix. The difference is where the bias comes from. Ridge and lasso bias is global and lives in coefficient space: one penalty shrinks every coefficient. KNN bias is local and geometric, the gap between $f(x_0)$ and the average of f over a neighborhood, so it depends on the curvature of f at x_0 and on the local design density. The same k can be nearly unbiased in a dense flat region and badly biased in a sparse one, with no clean analogue in a single λ .

Question 2: Compare KNN with lasso

Part (a)

The Week 4 objective with its $\frac{1}{2n}$ factor is exactly `sklearn`'s parameterization with `alpha` = λ , so the grid transfers without rescaling. Centering and scaling are re-estimated in every fold from that fold's training portion with divisor its sample size, the intercept is the fold-training response mean and is never penalized, and after λ is chosen the model is refit on all 400 training observations with those transformations applied unchanged to the test data.

► Show the workflow

```
seed = 433; 41 penalties from 1e-04 to 1e+00
```

► Show the workflow

setting	Setting 1	Setting 2
Var(y) on test	0.8491	1.4191
5NN test MSE	0.5823	0.1675
lasso test MSE	0.2768	0.2095
selected lambda	0.0316	0.0200
nonzero lasso slopes	7.0000	6.0000
5NN MSE / Var(y)	0.6858	0.1180
lasso MSE / Var(y)	0.3260	0.1476

The two methods swap places. With independent covariates the lasso wins by better than a factor of two; at correlation 0.8 the 5NN fit is better. The last two rows matter because $\text{Var}(y)$ differs between settings, so raw MSEs are not comparable across them: on the ratio scale KNN improves from about 0.69 to 0.12 of the response variance while the lasso moves only from 0.33 to 0.15.

Part (b)

► Show the workflow

```

setting
mean distance to 5th neighbor      Setting 1  Setting 2
mean distance to a random training point  7.7206  6.9351
ratio                               0.7529  0.4252

uniform-cube reference, side length for k/n = 5/400 in p = 30: 0.864
first six lasso slopes, Setting 1: [ 0.438  0.573 -0.018  0.   -0.   0.   ]
first six lasso slopes, Setting 2: [ 0.418  0.59   0.    0.   -0.   0.   ]

```

In Setting 1 the 30 coordinates are 30 separate directions and a neighborhood holding five of 400 points is local in none of them: $\ell = (k/n)^{1/p} = (5/400)^{1/30} \approx 0.86$ says it must span about 86% of each coordinate's range, and the design agrees, with the fifth neighbor about 75% as far as a random training point. Averaging over that is nearly averaging over the whole sample, the Q1 bias mechanism at an extreme. In Setting 2 the covariance $0.2I + 0.811^T$ has one eigenvalue of 24.2 and 29 of 0.2, so the cloud is essentially one direction with thin residual spread and its intrinsic dimension is far below 30. The fifth-neighbor distance drops to about 43% of a typical pairwise distance, and points close in X are now genuinely close in the coordinates driving the response: correlation helps KNN by lowering the effective dimension the distance must cover.

The lasso instead weights covariates by estimated coefficients and zeroes most of them: only six or seven of 30 slopes survive, so the 27 irrelevant ones cost little beyond estimation variance. Euclidean distance has no such mechanism, giving every coordinate weight one whether or not the response uses it, so those 27 consume the same share of the distance budget as the three that matter. The methods also meet correlation in opposite ways: it rescues KNN, but for the lasso it makes the design ill-conditioned and the choice among correlated columns unstable.

Part (c)

► Show the workflow

```

best linear coefficient on X2 = 0.6065; residual variance 0.0645
Var(-0.3 X3^2) = 0.1800, and Cov(X3, X3^2) = 0 for a standard normal
best possible linear-model test MSE, Setting 1 = 0.2545; lasso achieved 0.2768

```

The lasso fits linear terms to a mean function containing a sine and a square, so part of its error is approximation error no penalty can remove, and the arithmetic above sizes it. Since $\text{Cov}(X_3, X_3^2) = 0$ for a standard normal, the $-0.3X_3^2$ term is invisible to any linear model in these covariates and contributes its full variance 0.18. The sine is partly linear: the best coefficient on X_2 is $e^{-1/2} \approx 0.607$ and the fitted slope 0.573 is

that mildly shrunk, leaving 0.064. With $\sigma^2 = 0.01$ the floor is about 0.254 and the lasso achieved 0.277. It is already at its ceiling, so its Setting 1 advantage is not evidence that it has learned the truth; KNN has no such ceiling and in Setting 2 beats that linear floor. The table is therefore not a clean contest between two estimators but one method paying a dimension penalty against another paying an approximation penalty, with each setting changing only one of the two.

No general claim follows, in either direction. This is one dataset per setting with one n , one p , one correlation and one mean function, and with no repetition the differences carry no standard error. The ranking already flips inside this single experiment, which is itself the argument against “one method is always better”. “Correlation always helps KNN” fails too, because what helped was the alignment between the signal and the high-variance direction of the design. Had the response depended on a low-variance contrast such as $X_1 - X_2$, the same correlation would have compressed exactly the direction the response needs, and equicorrelated covariates would have hurt. The useful statement is about effective dimension and alignment, not correlation as such.

Question 3: Observed dimension and latent dimension

Part (a)

► Show the workflow

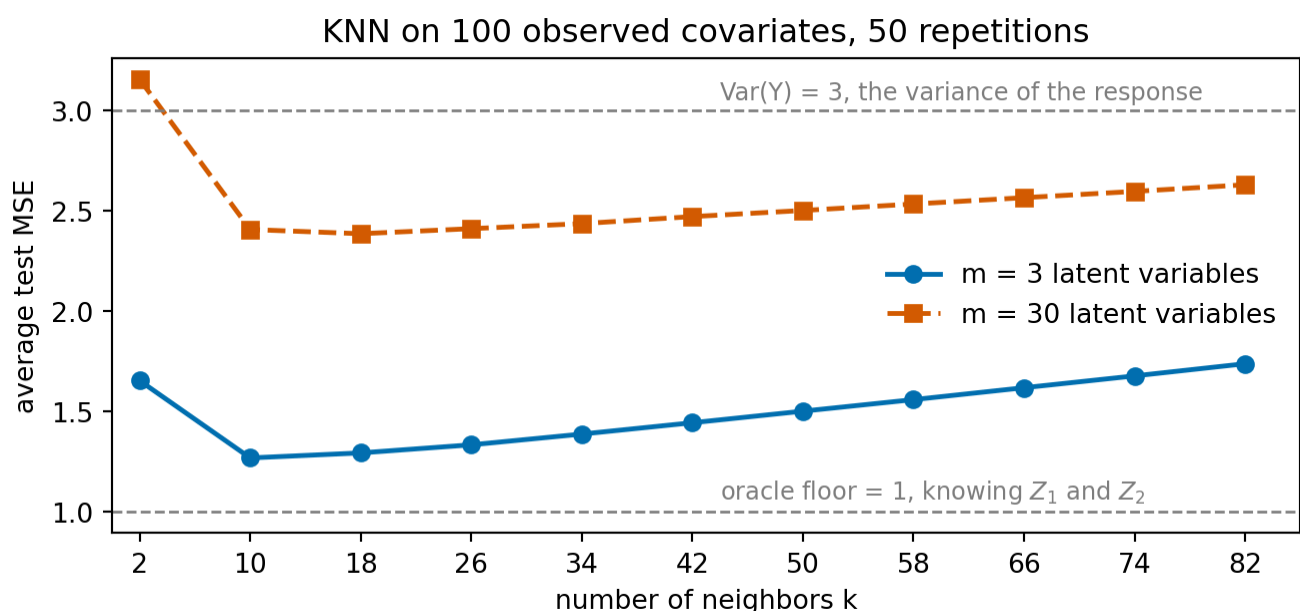
```
seed = 434; k = [2, 10, 18, 26, 34, 42, 50, 58, 66, 74, 82]
m = 3: test MSE by k 1.73 1.34 1.29 1.31 1.33 1.41 1.44 1.48 1.53 1.59 1.63
m = 30: test MSE by k 3.14 2.52 2.59 2.54 2.54 2.59 2.65 2.73 2.77 2.80 2.82
```

Part (b)

► Show the workflow

```
m = 3: average test MSE by k 1.65 1.27 1.29 1.33 1.39 1.44 1.50 1.56 1.62 1.68 1.74 | smallest 1.267 at k = 10
m = 30: average test MSE by k 3.16 2.41 2.39 2.41 2.44 2.47 2.50 2.53 2.57 2.60 2.63 | smallest 2.386 at k = 18
```

► Show the workflow



Average test MSE over 50 repetitions, one curve per latent dimension.

Both curves fall sharply from $k = 2$ and then rise slowly, but at completely different levels: the best average test MSE is 1.27 for $m = 3$ and 2.39 for $m = 30$, against $\text{Var}(Y) = 3$ and an oracle floor of 1, even though both settings have 100 observed covariates and a response depending on two latent variables.

Part (c)

► Show the workflow

```
seed = 435
m              3              30
corr(observed dist, relevant latent dist)  0.7928  0.2124
E||A(Zi-Zj)||^2      800.0000 8000.0000
E||eta_i-eta_j||^2    200.0000 200.0000
relevant share of E||Xi-Xj||^2      0.5333  0.0650
eta share of E||Xi-Xj||^2      0.2000  0.0244
```

The response is $Z_{i1} + Z_{i2} + \epsilon_i$ in both settings, so the function KNN must recover is equally simple; what differs is how much of the distance is spent on the two coordinates that matter. The columns of A are independent with variance $4/3$ per entry, so $E\|A(Z_i - Z_j)\|^2 = 2\|A\|_{F^2}^2$, of which the first two latent coordinates carry a $2/m$ share, about 533 either way. Hence the table: the relevant share of a squared distance falls from about 0.53 to 0.065, and the correlation between an observed distance and the distance in (Z_1, Z_2) from about 0.79 to 0.21. At $m = 3$ the nearest observed neighbors really are neighbors in the coordinates generating Y ; at $m = 30$ they mostly match on 28 irrelevant latent directions. The observed dimension is 100 either way and says nothing about which regime you are in, which is also why the lecture's 256-pixel digits work: those pixels have low intrinsic dimension.

η_i does two things, worth separating. It blurs the ranking of near points, adding $E\|\eta_i - \eta_j\|^2 = 200$ to every expected squared distance however close two points are in latent space, which is why even the $m = 3$ curve bottoms out near 1.27 rather than at the oracle floor of 1. But against first intuition it is not what makes $m = 30$ worse: its share of a squared distance is about 0.20 at $m = 3$ and only 0.02 at $m = 30$, since signal energy grows with m while the noise does not. The $m = 30$ setting has more signal in X and still predicts worse, so the damage comes from the 28 irrelevant latent directions, not the additive noise. Finally, AZ_i lies in the column space of A , of dimension m almost surely, but η_i is a nonsingular 100-dimensional Gaussian with a component outside it with probability one: $\text{Var}(X_i) = AA^T + I_{100}$ is full rank, its smallest $100 - m$ eigenvalues equal to 1. The cloud is an m -dimensional pancake thickened by one standard deviation in every direction; exact confinement would need $\eta_i = 0$, so low intrinsic dimension here is approximate, as for real images.

Question 4: Classify handwritten digits

Part (a)

The classifier forms distances from array arithmetic only, a difference, a square or absolute value, a sum and for Euclidean a square root: no distance routine and no KNN routine is called. Both tie rules follow from the choice of primitive. `np.argsort(kind="stable")` keeps the original order among equal distances, so a distance tie goes to the smaller training row number; `np.bincount(labels, minlength=10).argmax()` returns the first of several equal maxima over bins ordered 0 to 9, so a voting tie goes to the smallest digit.

► Show the workflow

```
used: train (1000, 256), test (500, 256); pixels in [-1, 1]
training label counts: [213, 120, 139, 70, 66, 47, 104, 75, 98, 68]
```

Part (b)

► Show the workflow

euclidean: error 0.1220 (61 of 500); true 2 as 0 (7x), true 9 as 7 (4x), true 5 as 3 (4x), true 5 as 0 (4x), true 4 as 9 (4x)
 manhattan: error 0.1380 (69 of 500); true 2 as 0 (6x), true 9 as 7 (5x), true 5 as 0 (5x), true 8 as 0 (4x), true 5 as 3 (4x)

► Show the workflow

Euclidean distance (rows = predicted, columns = true)

	true 0	true 1	true 2	true 3	true 4	true 5	true 6	true 7	true 8	true 9
pred 0	118	0	7	1	0	4	2	0	2	0
pred 1	0	66	0	0	2	0	0	0	0	1
pred 2	0	0	39	1	0	0	2	0	0	1
pred 3	0	0	0	23	0	4	0	0	2	0
pred 4	0	0	1	0	27	1	1	1	0	1
pred 5	0	0	0	1	0	13	0	0	0	0
pred 6	0	2	0	0	0	1	41	0	2	0
pred 7	0	0	0	0	1	1	0	34	0	4
pred 8	0	0	3	2	0	2	0	0	37	0
pred 9	1	0	0	1	4	0	0	1	1	41

► Show the workflow

Manhattan distance (rows = predicted, columns = true)

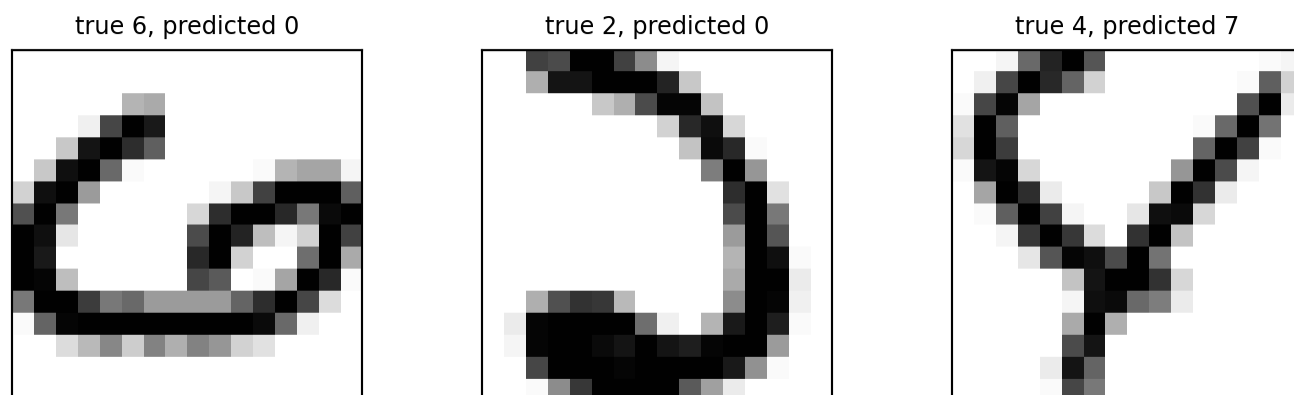
	true 0	true 1	true 2	true 3	true 4	true 5	true 6	true 7	true 8	true 9
pred 0	118	0	6	1	0	5	3	0	4	0
pred 1	0	67	2	0	2	0	1	0	0	1
pred 2	0	0	35	0	0	0	2	0	0	0
pred 3	0	0	1	24	0	4	0	0	2	0
pred 4	0	0	1	1	27	1	0	1	0	1
pred 5	0	0	0	1	0	11	0	0	0	0
pred 6	0	1	0	0	0	2	40	0	2	0
pred 7	0	0	2	1	1	0	0	34	0	5
pred 8	0	0	2	0	0	3	0	0	34	0
pred 9	1	0	1	1	4	0	0	1	2	41

Both errors are well above the roughly 5% of the lecture, as expected from using 1,000 of the 7,291 training images. The confusion patterns are nearly identical across the two distances. The heaviest cell in both is a true 2 read as 0, and the other recurring pairs are a true 9 read as 7, a true 5 read as 3 or as 0, a true 4 read as 9 under Euclidean and a true 8 read as 0 under Manhattan. These share most of their ink once stroke identity is discarded: a round 2 and a 0 both fill the frame with one large curve, 9 and 7 differ mainly in whether the upper loop closes, 5 and 3 share the lower bowl, 4 and 9 in whether the top is closed. Row 0 is the widest in both, reflecting that 0 is by far the most frequent training label here, 213 of 1,000, so an ambiguous image has many more chances to find five nearby zeros.

Part (c)

► Show the workflow

```
test row 3: true 6, predicted 0, neighbor labels [0, 0, 6, 0, 0], votes {0: 4, 6: 1}
test row 12: true 2, predicted 0, neighbor labels [0, 3, 2, 2, 0], votes {0: 2, 2: 2, 3: 1}
test row 13: true 4, predicted 7, neighbor labels [7, 4, 8, 7, 7], votes {4: 1, 7: 3, 8: 1}
```



Three test images misclassified under Euclidean distance.

► Show the workflow

```
wrong under both: 58; Euclidean only: 3; Manhattan only: 11; the two fits disagree on 22 of 500
```

Each mistake fails for a reason visible in the image. The first is a 6 written as one wide sweeping stroke with a long flat base, so its outer contour closes into a ring filling the frame; the small terminal loop that makes it a 6 covers a handful of pixels while the ring dominates a sum of 256 squared differences, and four of its five neighbors are fat zeros. The second is a very round 2 with a curled base, whose neighbors split two zeros, two twos and one three: the vote is tied 2–2 and the prediction is settled by the tie rule taking the smallest digit, so this is not really a distance failure. The third is an open-top 4 in a “y” style dominated by one long diagonal, the defining stroke of a 7, and three of its neighbors are sevens. The common thread is that Euclidean distance on raw pixels compares total ink overlap with no notion of which stroke carries the digit’s identity, so a small distinguishing feature contributes little to a sum over 256 coordinates. None of the three is a clean, well-centered digit misread for no visible reason.

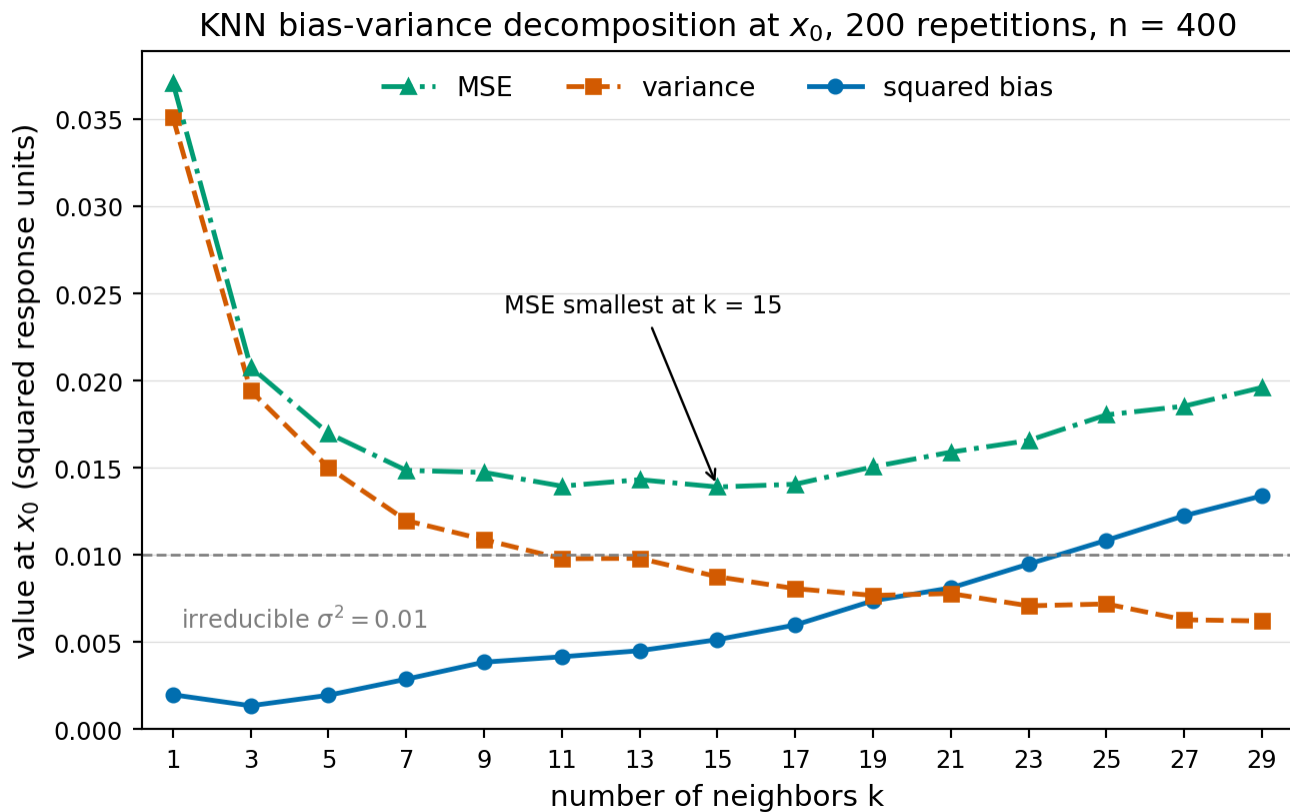
Euclidean is slightly better overall, 12.2% against 13.8%, but with 500 test points the standard error of an error rate near 0.13 is about 0.015, so a gap of 8 images is not strong evidence of a real difference. **Can two distances have similar total error while erring on different images? Yes, and these predictions show it.** Of 500 images, 58 are misclassified by both, 3 by Euclidean only and 11 by Manhattan only, so the error sets are neither identical nor nested: 14 images are decided correctly by exactly one of the two. The fits return different predictions on 22 of 500, agreeing 95.6% of the time, so most of the shared error rate comes from the same genuinely ambiguous images while a small set is settled by the choice of metric. A single error rate is a marginal summary, which is why the confusion matrix and the paired comparison belong next to it.

Question 5: Create a plot-formatting skill

Parts (a) and (b)

The skill is installed at [.claude/skills/stat432-plot-format/SKILL.md](#). Its description names STAT 432 homework plots and requires being asked for by name, so it does not fire on its own during other work; the rules cover size and margins, titles, axes, labels and legends, colors and sizes, for either R or Python and with no reference to a particular question. It was then called explicitly on the Q1 figure, whose default version appears in Q1 and its revision below.

► Show the workflow



Question 1 figure after applying the plot-formatting skill.

I like most of the changes, for reasons I can name rather than taste. Ticking the actual grid makes “MSE smallest at $k = 15$ ” checkable on the figure, where the default left the reader to work out that only odd k appear. The colorblind-safe palette paired with three line styles lets the series survive both a red-green deficiency and a grayscale printout, which the default palette, relying on color alone, does not. Annotating the minimum and drawing the σ^2 line put the two numbers the text discusses onto the figure. Checking the rules one at a time also caught four things I had done wrong on the first pass: a dotted rather than dashed reference line, horizontal limits padded out to 33 only to fit a label, a legend listed bottom-up while the curves run MSE, variance, squared bias from the top, and a vertical axis with no scale. The last is the interesting case: I first assumed the rule could not apply when three quantities share an axis, but all three are in squared response units. One rule genuinely did not fit, since labeling each series at the end of its curve is impossible when all three converge at the right edge, so I used the frameless legend the same rule offers instead.

Part (c)

The final `SKILL.md`:

```
---
name: stat432-plot-format
description: Improve the formatting of a plot in a STAT 432 homework report, in R or Python. Use this skill o
---

# STAT 432 Homework Plot Formatting

Change presentation only. Never change the data, the model, or the plotted quantities, and
never silently drop a series.

**Size and margins.** One panel on roughly 7 x 4.5 inches. Leave room for the labels with
`plt.tight_layout()` or `par(mar = c(4.5, 4.5, 3, 1))`. Let the limits follow the data, and
extend one only to reach a reference value such as zero.
```

****Title.**** One short line saying what the figure shows, not the question number and not a restatement of the axis names. Put the settings the reader needs, such as sample size, there.

****Axes.**** Label both with the quantity and its scale or unit. Tick the values a reader has to look up: for a tuning parameter on a short grid, tick the grid itself. Use a log scale when the values span more than about two orders of magnitude, and say so in the label.

****Labels and legend.**** With at most four series, label them at the end of each curve or use a legend inside the panel with no frame. Order the entries to match the visual order of the series. Annotate directly any value the text refers to, such as the minimum of a curve.

****Colors.**** Colorblind-safe, at most four per panel; Okabe and Ito (`#0072B2`, `#D55E00`, `#009E73`, `#CC79A7`) is a safe default. Never encode a series by color alone: give each its own line type or marker so the figure survives grayscale. White panel, at most a light horizontal grid.

****Sizes.**** Base font about 11 points and tick labels no smaller than 9. Line width about 1.8 for a series and 1 for a reference line, which should be thin, gray, dashed, and labeled in place.

****Do not**** use a second y-axis, rely on red against green, let a grid compete with the series, or rescale the plotted values to make a curve look better behaved.

The description does two jobs: it scopes the skill narrowly enough that it is not a general “make plots pretty” tool, and it requires an explicit request, because a formatting skill firing on its own would restyle figures during unrelated work and leave it unclear which version of a plot a report contains. I kept the rules short and checkable, since “use clear labels” cannot be verified while “tick the grid itself” can. The colors rule is the one I would defend most: requiring a redundant encoding through line type or marker is what makes a figure survive grayscale printing and color vision deficiency, a correctness property rather than taste. The line forbidding changes to the plotted values is there because the one real risk in handing a figure to an agent is that it improves the appearance by changing the content. Using it showed that the two rules with the least room for judgment, axis units and end-of-curve labels, fit this figure worst: a short skill is easy to follow literally, which is exactly when a rule stated without its purpose produces a worse figure. If I revise it again I will give each rule a clause saying what it is for, so it can be overridden deliberately rather than satisfied mechanically.

